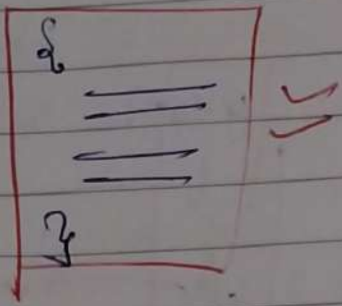
# Java I/O

Standard I/O



Types of I/O
- Console I/O
- File I/O
- Network I/O
- Memory I/O

* Console I/O

system.out.println("Hellow");

```
class PointStream {
    void println() {

    }
}
```

```
class system {
    static  x  out;
}
```

System.out.println("Hello");

```
class System {
}
```

Java.lang

print Stream → println() new line
→ print() No new line
→ printf() formatted o/p

```
class System {
    printStream out;

}
```

```
System s = new System();
s.out.println("Hello");
```

* Input

```
class System {
    printStream out;     // O/P
    printStream err;
    InputStream in;
}
```

System.in → Input-Stream

Streams → flow of Data

Input Stream → Data flow INTO Program
Output Stream → Data flow OUT of Program

Java classes (Abstract)        read()
                               write()

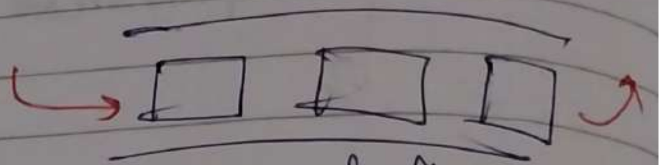
Input Stream → read()
output Stream → write();

file    console    N/w

System.in → Input Stream
System.out, System.err → Output Stream → printStream

Java I/O → Stream Based

- console
- file
- memory
- N/W



Data flow

All of them are "stream of Bytes"

(System.in) Input Stream (Abstract) → read();
- → File Input Stream → read() {}
- → Byte Array Input Steam → read () {}.
- → Buffered Input Steam → read() {}
- → DataInput Steam → read() {}

Output Steam (Abstract) → write();
- → File Output Steam → write() {}
- → Byte Array Output Steam → write () {}
- → Buffered Output Steam → write () {}
- (System.out) (System.err) → Print Steam → write() {}

System.in → Input Steam
- → keyboard.

Input Steam → read()
- → Steam of Bytes

System.out.println();
System.in.read();

I/P : A

ASCII

65

$\boxed{65}$

Input Steam

- int data = System.in.read();
System.out.println((char) data);

read() ⟶ one byte at a time

'A [ditya] → Input Buffer

A

Aditya

Buffer

[A] [d] [i] [t] [y] [a] [\n]

[65] [100] [105] [116] [121] [97] [10]

System.in.read():

// Aditya \n

```
String s = "";
int c;
while (c != '\n'){
    c = System.in.read();
    s += (char)c;
}
```

cin>>

scanf()

stream of characters

Reader (Abstract)
- → Buffered Reader
- → Input Stream Reader
- → File Reader

Buffered Reader → Buffer

1 char
1 char
1 char
⋮

1000 OS call

Keybord → OS Buffer → Program

A → OS A → AD
D

keyboard → OS Buffer → Java Buffer → Program

ADITYA → ADITYA

Buffered Reader
- Read chunk of characters from OS Buffer
- Store it in memory
- Give them to program when required

Standard I/P → Input Stream

System.in

Buffered Reader → Stream of char

Input Stream → Stream of Bytes

$\boxed{\text{Input Stream Reader}}$

* Input Stream Reader

byte Stream ⟶ character stream

Input Stream Reader Isr = new ______ (Input Stream (System.in)

Buffered Reader

Buffered Reader br = new ______ (Isr)

: br.readline();

limitations of Buffered Reader : String s = br.readLine();
int i = Integer.parseInt (S);

* Java - 1.5
→ Scanner
1) simplify (Input)
2) int, double, boolean, string

class Scanner {
=====
}

→ java.io (X)
java.util

InputStream Reader

Scanner → Readable → Reader

Scanner
↓
Input Stream (System.in)
↑
Input Stream Reader
↑
Buffered Reader

* Creating Scanner

Scanner sc = new Scanner( );
keyboard ↗ System.in;

File

Scanner sc = new Scanner( );
new File("Sample.txt");

String

= new Scanner(" 10,20,30 ");
sc. readLine();

Scanner ⟶ Tokenization

Hello | I | am | Aditya

Methods of Scanner
↳ next()
nextLine()
nextInt()